🎓 PCN Lab Exam: Minimal Code Study & Cheat Sheet

> **Strictly tailored to Teacher's Exam Blueprint:**
>
> - **Language**: Pure **C** for Sockets | Pure **C++** for NS-3. (No Python).
> - **Teacher's Scope**: *"NS3 both wired and wireless. Sockets: TCP, UDP, timers, retransmissions, CRC, ARP, DNS. Metrics: latency, throughput, RTT, PDR. Web page download from local HTTP server."*
> - **Goal**: Ultra-minimal, memorizable code blocks (~30-50 lines) that you can write on paper or code in 10 minutes.

📑 Quick Navigation

1. Core Formula & Metric Snippets (Copy into any program)
2. Socket Programming in C (Assignments 2, 3, 5, 6 + ARP)
    - Program 1: TCP Client-Server with RTT & Throughput (Ass 2)
    - Program 2: UDP Client-Server with Timer & Retransmissions
    - Program 3: HTTP Web Client in C (Local Server Download) (Ass 3)
    - Program 4: DNS Simulation over UDP with Caching (Ass 5)
    - Program 5: Error Detection — 8-Bit Checksum & CRC (Ass 6)
    - Program 6: ARP Simulation over Sockets
3. NS-3 Network Simulations in C++ (Assignments 7, 8, 9, 10, 11)
    - Program 7: Point-to-Point UDP Latency vs Throughput (Ass 7)
    - Program 8: TCP & UDP Bottleneck Router with DropTail Queue (Ass 8)
    - Program 9: TCP Congestion Control (Reno vs Cubic vs BBR) (Ass 9)
    - Program 10: Routing Protocols (RIP / OSPF / AODV) & Link Failure (Ass 10)
    - Program 11: Wireless Network with Mobility & PDR (Ass 11)

⚡ 0. The Universal Metrics Snippets

Whenever the question asks for **RTT, Latency, or Throughput in C**, paste this 4-line snippet:

```c
#include <time.h>

struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);

// ... [network send / recv calls] ...

clock_gettime(CLOCK_MONOTONIC, &end);
double rtt_ms = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_nsec - start.tv_nsec) / 1e6;
double throughput_kbps = (bytes_received * 8.0) / (rtt_ms); // bits / ms = kbps

printf("RTT / Latency : %.2f ms\n", rtt_ms);
printf("Throughput    : %.2f Kbps\n", throughput_kbps);
```

Whenever the question asks for **Throughput, Delay, or PDR in NS-3 (C++)**, use FlowMonitor:

```cpp
#include "ns3/flow-monitor-module.h"

FlowMonitorHelper flowmon;
Ptr<FlowMonitor> monitor = flowmon.InstallAll();

Simulator::Stop(Seconds(10.0));
Simulator::Run();

monitor->CheckForLostPackets();
std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats();
for (auto const& [id, s] : stats) {
    double throughput = s.rxBytes * 8.0 / (10.0 * 1e6); // Mbps
    double pdr = (double)s.rxPackets / (s.txPackets) * 100.0;
    double avg_delay = s.delaySum.GetSeconds() / s.rxPackets * 1000.0; // ms
    std::cout << "Flow " << id << " | Throughput: " << throughput << " Mbps | PDR: "
              << pdr << "% | Delay: " << avg_delay << " ms\n";
}
Simulator::Destroy();
```

🌐 Part 1: Socket Programming in C

Program 1: TCP Client-Server with RTT & Throughput

Problem: Implement client-server communication using TCP sockets in C and measure RTT and data throughput.

`tcp_server.c` (~25 lines)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in saddr = { .sin_family = AF_INET, .sin_port = htons(8080), .sin_addr.s_ad

    bind(sfd, (struct sockaddr*)&saddr, sizeof(saddr));
    listen(sfd, 5);
    printf("Server listening on port 8080...\n");

    int cfd = accept(sfd, NULL, NULL);
    char buf[1024];
    int n = read(cfd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("Received from client: %s\n", buf);

    char reply[] = "ACK: Student Record 3122247001062 - Simiyon Verified";
    write(cfd, reply, strlen(reply));

    close(cfd);
    close(sfd);
    return 0;
}
```

`tcp_client.c` (~32 lines)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <arpa/inet.h>

int main() {
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in saddr = { .sin_family = AF_INET, .sin_port = htons(8080) };
    inet_pton(AF_INET, "127.0.0.1", &saddr.sin_addr);

    connect(sfd, (struct sockaddr*)&saddr, sizeof(saddr));

    char req[] = "Query: RollNo 3122247001062";
    char buf[1024];
    struct timespec t1, t2;

    clock_gettime(CLOCK_MONOTONIC, &t1);
    write(sfd, req, strlen(req));
    int n = read(sfd, buf, sizeof(buf) - 1);
    clock_gettime(CLOCK_MONOTONIC, &t2);
    buf[n] = '\0';

    double rtt = (t2.tv_sec - t1.tv_sec) * 1000.0 + (t2.tv_nsec - t1.tv_nsec) / 1e6;
    double throughput = (n * 8.0) / rtt; // Kbps

    printf("Server Reply : %s\n", buf);
    printf("RTT Latency  : %.3f ms\n", rtt);
    printf("Throughput   : %.2f Kbps\n", throughput);

    close(sfd);
    return 0;
}
```

Compile & Run:

```
gcc tcp_server.c -o s && gcc tcp_client.c -o c
./s & ./c
```

Program 2: UDP Client-Server with Timer & Retransmissions

Teacher's Note: "timers, retransmissions in UDP sockets"

`udp_server.c` (~20 lines)

```c
#include <stdio.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int sfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in saddr = { .sin_family = AF_INET, .sin_port = htons(9090), .sin_addr.s_ad
    bind(sfd, (struct sockaddr*)&saddr, sizeof(saddr));

    char buf[128];
    struct sockaddr_in caddr;
    socklen_t clen = sizeof(caddr);

    printf("UDP Server listening on 9090...\n");
    int n = recvfrom(sfd, buf, sizeof(buf)-1, 0, (struct sockaddr*)&caddr, &clen);
    buf[n] = '\0';
    printf("Received packet: %s\n", buf);

    sendto(sfd, "ECHO_ACK", 8, 0, (struct sockaddr*)&caddr, clen);
    close(sfd);
    return 0;
}
```

`udp_client_timer.c` (~38 lines — Implements Socket Timeout &

Retry)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>

int main() {
    int sfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in saddr = { .sin_family = AF_INET, .sin_port = htons(9090) };
    inet_pton(AF_INET, "127.0.0.1", &saddr.sin_addr);

    // Set Socket Receive Timeout: 2 seconds
    struct timeval tv = { .tv_sec = 2, .tv_usec = 0 };
    setsockopt(sfd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

    char msg[] = "DATA_FRAME_1";
    char buf[128];
    int max_retries = 3, attempts = 0, success = 0;

    while (attempts < max_retries && !success) {
        attempts++;
        printf("[Attempt %d] Sending: %s\n", attempts, msg);
        sendto(sfd, msg, strlen(msg), 0, (struct sockaddr*)&saddr, sizeof(saddr));

        int n = recvfrom(sfd, buf, sizeof(buf)-1, 0, NULL, NULL);
        if (n < 0) {
            printf(" -> Timeout! No ACK received. Retransmitting...\n");
        } else {
            buf[n] = '\0';
            printf(" -> Success! Received ACK: %s\n", buf);
            success = 1;
        }
    }

    if (!success) printf("Failed: Max retransmissions reached.\n");
    close(sfd);
    return 0;
}
```

Program 3: HTTP Web Client in C (Local Web Server Download)

Teacher’s Note: “Also web page download from local http webserver”

`http_client.c` (~35 lines)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <arpa/inet.h>

int main() {
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in saddr = { .sin_family = AF_INET, .sin_port = htons(8000) };
    inet_pton(AF_INET, "127.0.0.1", &saddr.sin_addr);

    if (connect(sfd, (struct sockaddr*)&saddr, sizeof(saddr)) < 0) {
        perror("Connect failed (is python3 -m http.server 8000 running?)");
        return 1;
    }

    char req[] = "GET /index.html HTTP/1.1\r\nHost: localhost\r\nConnection: close\r\n\r\n";
    struct timespec t1, t2;

    clock_gettime(CLOCK_MONOTONIC, &t1);
    write(sfd, req, strlen(req));

    char buf[4096];
    int total_bytes = 0, n;
    FILE *fp = fopen("downloaded.html", "w");

    while ((n = read(sfd, buf, sizeof(buf))) > 0) {
        fwrite(buf, 1, n, fp);
        total_bytes += n;
    }
    clock_gettime(CLOCK_MONOTONIC, &t2);
    fclose(fp);
    close(sfd);

    double duration = (t2.tv_sec - t1.tv_sec) * 1000.0 + (t2.tv_nsec - t1.tv_nsec) / 1e6;
    printf("Downloaded : %d bytes into downloaded.html\n", total_bytes);
    printf("Total Time : %.2f ms\n", duration);
    printf("Throughput : %.2f KB/s\n", (total_bytes / 1024.0) / (duration / 1000.0));
```

```
    return 0;
}
```

Program 4: DNS Simulation over UDP with Caching

Requirements: DNS Dictionary table, Cache lookup, NXDOMAIN, Timeout.

`dns_server.c` (~28 lines)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

struct DNS { char domain[32]; char ip[32]; } table[] = {
    {"google.com", "142.250.190.46"},
    {"ssn.edu.in", "103.109.8.10"},
    {"github.com", "140.82.121.4"}
};

int main() {
    int sfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in saddr = { .sin_family = AF_INET, .sin_port = htons(5053), .sin_addr.s_ad
    bind(sfd, (struct sockaddr*)&saddr, sizeof(saddr));

    char query[32], reply[32];
    struct sockaddr_in caddr;
    socklen_t clen = sizeof(caddr);

    printf("DNS Server running on port 5053...\n");
    while (1) {
        int n = recvfrom(sfd, query, sizeof(query)-1, 0, (struct sockaddr*)&caddr, &clen);
        query[n] = '\0';
        strcpy(reply, "NXDOMAIN");

        for (int i = 0; i < 3; i++) {
            if (strcmp(table[i].domain, query) == 0) {
                strcpy(reply, table[i].ip);
                break;
            }
        }
        sendto(sfd, reply, strlen(reply), 0, (struct sockaddr*)&caddr, clen);
    }
    return 0;
}
```

`dns_client.c` (~40 lines)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

struct Cache { char domain[32]; char ip[32]; } cache;
int cache_valid = 0;

int main() {
    int sfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in saddr = { .sin_family = AF_INET, .sin_port = htons(5053) };
    inet_pton(AF_INET, "127.0.0.1", &saddr.sin_addr);

    struct timeval tv = { .tv_sec = 2, .tv_usec = 0 };
    setsockopt(sfd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

    char domain[32], ip[32];
    while (1) {
        printf("Enter domain (or exit): ");
        scanf("%s", domain);
        if (strcmp(domain, "exit") == 0) break;

        // 1. Check Local cache
        if (cache_valid && strcmp(cache.domain, domain) == 0) {
            printf("[CACHE HIT] %s -> %s (0.01 ms)\n", domain, cache.ip);
            continue;
        }

        // 2. Query Server
        sendto(sfd, domain, strlen(domain), 0, (struct sockaddr*)&saddr, sizeof(saddr));
        int n = recvfrom(sfd, ip, sizeof(ip)-1, 0, NULL, NULL);
        if (n < 0) {
            printf("[TIMEOUT] Server did not respond.\n");
        } else {
            ip[n] = '\0';
            printf("[SERVER RESOLVED] %s -> %s\n", domain, ip);
            if (strcmp(ip, "NXDOMAIN") != 0) {
                strcpy(cache.domain, domain);
                strcpy(cache.ip, ip);
```

```
                cache_valid = 1;
            }
        }
    }
    close(sfd);
    return 0;
}
```

Program 5: Error Detection — 8-Bit Checksum & CRC

Checksum Algorithm (~22 lines)

```
#include <stdio.h>

unsigned char compute_checksum(unsigned char *data, int len) {
    unsigned int sum = 0;
    for (int i = 0; i < len; i++) sum += data[i];
    while (sum >> 8) sum = (sum & 0xFF) + (sum >> 8); // End-around carry
    return (unsigned char)(~sum); // 1's complement
}

int main() {
    unsigned char msg[] = "HELLO";
    unsigned char cs = compute_checksum(msg, 5);
    printf("Data: %s | Checksum: 0x%02X\n", msg, cs);

    // Receiver Verification
    unsigned char rx[6] = "HELLO";
    rx[5] = cs;
    unsigned int rx_sum = 0;
    for (int i = 0; i < 6; i++) rx_sum += rx[i];
    while (rx_sum >> 8) rx_sum = (rx_sum & 0xFF) + (rx_sum >> 8);

    if ((unsigned char)(~rx_sum) == 0) printf("Clean: NO ERROR (Accepted)\n");
    else printf("Corrupted: ERROR DETECTED\n");
    return 0;
}
```

CRC Algorithm (Modulo-2 Division) (~28 lines)

```c
#include <stdio.h>
#include <string.h>

void crc(char *data, char *gen, char *rem) {
    int dlen = strlen(data), glen = strlen(gen);
    char temp[128];
    strcpy(temp, data);
    for (int i = 0; i < glen - 1; i++) strcat(temp, "0");

    for (int i = 0; i < dlen; i++) {
        if (temp[i] == '1') {
            for (int j = 0; j < glen; j++)
                temp[i + j] = (temp[i + j] == gen[j]) ? '0' : '1';
        }
    }
    strcpy(rem, temp + dlen);
}

int main() {
    char data[] = "1101001110", gen[] = "10011", rem[16];
    crc(data, gen, rem);
    printf("Message   : %s\nGenerator : %s\nCRC Rem   : %s\n", data, gen, rem);
    printf("Transmitted Frame: %s%s\n", data, rem);
    return 0;
}
```

Program 6: ARP (Address Resolution Protocol) Simulation

Teacher's Note: "ARP simulation over sockets"

`arp_server.c` (~25 lines)

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

struct ARP { char ip[16]; char mac[18]; } arp_table[] = {
    {"192.168.1.1", "AA:BB:CC:DD:EE:01"},
    {"192.168.1.2", "AA:BB:CC:DD:EE:02"},
    {"192.168.1.3", "AA:BB:CC:DD:EE:03"}
};

int main() {
    int sfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in saddr = { .sin_family = AF_INET, .sin_port = htons(7070), .sin_addr.s_ad
    bind(sfd, (struct sockaddr*)&saddr, sizeof(saddr));

    char req_ip[16], reply[18];
    struct sockaddr_in caddr; socklen_t clen = sizeof(caddr);

    printf("ARP Resolver running on port 7070...\n");
    while (1) {
        int n = recvfrom(sfd, req_ip, sizeof(req_ip)-1, 0, (struct sockaddr*)&caddr, &clen);
        req_ip[n] = '\0';
        strcpy(reply, "MAC_NOT_FOUND");
        for (int i = 0; i < 3; i++) {
            if (strcmp(arp_table[i].ip, req_ip) == 0) { strcpy(reply, arp_table[i].mac); break;
        }
        sendto(sfd, reply, strlen(reply), 0, (struct sockaddr*)&caddr, clen);
    }
    return 0;
}
```

🚀 Part 2: NS-3 Simulations in C++

Every NS-3 script follows the identical **5-Step Structure**:

1. `NodeContainer nodes; nodes.Create(N);`
2. `PointToPointHelper p2p; ... NetDeviceContainer devices = p2p.Install(nodes);`
3. `InternetStackHelper stack; stack.Install(nodes);`
4. `Ipv4AddressHelper address; ... Ipv4InterfaceContainer interfaces = address.Assign(devices);`
5. `FlowMonitorHelper flowmon; ... monitor->GetFlowStats();`

Program 7: NS-3 Wired — P2P UDP Latency vs Throughput (Ass 7)

Requirements: 2 nodes, varying latency (2ms, 5ms, 10ms, 20ms), 2 ports (5000 & 6000).

`p2p_udp.cc` (~45 lines)

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

int main() {
    NodeContainer nodes;
    nodes.Create(2);

    PointToPointHelper p2p;
    p2p.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
    p2p.SetChannelAttribute("Delay", StringValue("5ms")); // Change to 2ms, 10ms, 20ms

    NetDeviceContainer devices = p2p.Install(nodes);

    InternetStackHelper stack;
    stack.Install(nodes);

    Ipv4AddressHelper address;
    address.SetBase("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer interfaces = address.Assign(devices);

    // App 1: Port 5000
    UdpServerHelper server1(5000);
    server1.Install(nodes.Get(1)).Start(Seconds(1.0));
    UdpClientHelper client1(interfaces.GetAddress(1), 5000);
    client1.SetAttribute("MaxPackets", UintegerValue(1000));
    client1.SetAttribute("Interval", TimeValue(MilliSeconds(10)));
    client1.Install(nodes.Get(0)).Start(Seconds(2.0));

    // App 2: Port 6000
    UdpServerHelper server2(6000);
    server2.Install(nodes.Get(1)).Start(Seconds(1.0));
    UdpClientHelper client2(interfaces.GetAddress(1), 6000);
    client2.SetAttribute("MaxPackets", UintegerValue(1000));
```

```
    client2.SetAttribute("Interval", TimeValue(MilliSeconds(10)));
    client2.Install(nodes.Get(0)).Start(Seconds(2.0));

    FlowMonitorHelper flowmon;
    Ptr<FlowMonitor> monitor = flowmon.InstallAll();

    Simulator::Stop(Seconds(10.0));
    Simulator::Run();

    for (auto const& [id, s] : monitor->GetFlowStats()) {
        std::cout << "Flow " << id << " Throughput: " << s.rxBytes * 8.0 / (10.0 * 1e6) << " Mb
    }
    Simulator::Destroy();
    return 0;
}
```

Program 8: NS-3 Wired — TCP vs UDP Bottleneck Router (Ass 8)

Requirements: 4 end nodes, 2 routers, 2 Mbps bottleneck link, DropTail queue.

`bottleneck.cc` (~50 lines)

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

int main() {
    NodeContainer nodes;
    nodes.Create(6); // n0, n1 (sources) -> r0 -> r1 (bottleneck) -> n2, n3 (destinations)

    PointToPointHelper access, bottleneck;
    access.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
    access.SetChannelAttribute("Delay", StringValue("2ms"));

    bottleneck.SetDeviceAttribute("DataRate", StringValue("2Mbps")); // 2, 5, 10 Mbps
    bottleneck.SetChannelAttribute("Delay", StringValue("10ms"));
    bottleneck.SetQueue("ns3::DropTailQueue<Packet>", "MaxSize", StringValue("50p"));

    NetDeviceContainer d_n0r0 = access.Install(nodes.Get(0), nodes.Get(2));
    NetDeviceContainer d_n1r0 = access.Install(nodes.Get(1), nodes.Get(2));
    NetDeviceContainer d_r0r1 = bottleneck.Install(nodes.Get(2), nodes.Get(3));
    NetDeviceContainer d_r1n2 = access.Install(nodes.Get(3), nodes.Get(4));
    NetDeviceContainer d_r1n3 = access.Install(nodes.Get(3), nodes.Get(5));

    InternetStackHelper stack;
    stack.Install(nodes);

    Ipv4AddressHelper ip;
    ip.SetBase("10.0.1.0", "255.255.255.0"); ip.Assign(d_n0r0);
    ip.SetBase("10.0.2.0", "255.255.255.0"); ip.Assign(d_n1r0);
    ip.SetBase("10.0.3.0", "255.255.255.0"); ip.Assign(d_r0r1);
    ip.SetBase("10.0.4.0", "255.255.255.0"); ip.Assign(d_r1n2);
    ip.SetBase("10.0.5.0", "255.255.255.0"); auto if_n3 = ip.Assign(d_r1n3);

    Ipv4GlobalRoutingHelper::PopulateRoutingTables();
```

```
    // TCP: Node 0 -> Node 3 (Port 9000)
    PacketSinkHelper tcpSink("ns3::TcpSocketFactory", InetSocketAddress(Ipv4Address::GetAny(),
    tcpSink.Install(nodes.Get(5)).Start(Seconds(0.5));
    BulkSendHelper tcpSource("ns3::TcpSocketFactory", InetSocketAddress(if_n3.GetAddress(1), 90
    tcpSource.Install(nodes.Get(0)).Start(Seconds(1.0));

    FlowMonitorHelper flowmon;
    Ptr<FlowMonitor> monitor = flowmon.InstallAll();

    Simulator::Stop(Seconds(10.0));
    Simulator::Run();

    for (auto const& [id, s] : monitor->GetFlowStats()) {
        std::cout << "Flow " << id << " Loss: " << s.lostPackets << " | Delay: "
                  << s.delaySum.GetSeconds() / s.rxPackets * 1000.0 << " ms\n";
    }
    Simulator::Destroy();
    return 0;
}
```

Program 9: NS-3 Wired — TCP Congestion Control Variants (Ass 9)

Requirements: Set TCP Reno, Cubic, or BBR in NS-3.

```cpp
#include "ns3/core-module.h"
// ... [Standard p2p setup] ...
using namespace ns3;

int main(int argc, char *argv[]) {
    std::string tcpVariant = "TcpReno"; // or "TcpCubic" or "TcpBbr"

    if (tcpVariant == "TcpReno")
        Config::SetDefault("ns3::TcpL4Protocol::SocketType", TypeIdValue(TcpReno::GetTypeId()))
    else if (tcpVariant == "TcpCubic")
        Config::SetDefault("ns3::TcpL4Protocol::SocketType", TypeIdValue(TcpCubic::GetTypeId())
    else if (tcpVariant == "TcpBbr")
        Config::SetDefault("ns3::TcpL4Protocol::SocketType", TypeIdValue(TcpBbr::GetTypeId()));

    // Install BulkSend / PacketSink and run FlowMonitor as shown in Program 8
}
```

Program 10: NS-3 Wired — Routing Protocols (Ass 10)

Requirements: Compare RIP, OSPF, AODV and simulate Link Failure.

```cpp
// 1. For OSPF:
Ipv4GlobalRoutingHelper::PopulateRoutingTables();

// 2. For RIP:
RipHelper rip;
InternetStackHelper stack;
stack.SetRoutingHelper(rip);
stack.Install(nodes);

// 3. For Link Failure at time 5.0s:
Simulator::Schedule(Seconds(5.0), &PointToPointChannel::SetAttribute,
                    channel, "Delay", StringValue("1000s")); // Effectively drops link
```

Program 11: NS-3 Wireless — Heterogeneous Wi-Fi with Mobility (Ass 11)

Teacher’s Note: “ns3 wireless, mobility, and PDR”

`wifi_mobility.cc` (~50 lines)

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/mobility-module.h"
#include "ns3/wifi-module.h"
#include "ns3/internet-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

int main() {
    NodeContainer staNodes, apNode;
    staNodes.Create(3);
    apNode.Create(1);

    YansWifiChannelHelper channel = YansWifiChannelHelper::Default();
    YansWifiPhyHelper phy;
    phy.SetChannel(channel.Create());

    WifiHelper wifi;
    wifi.SetStandard(WIFI_STANDARD_80211n);

    WifiMacHelper mac;
    Ssid ssid = Ssid("campus-wifi");

    mac.SetType("ns3::StaWifiMac", "Ssid", SsidValue(ssid));
    NetDeviceContainer staDevices = wifi.Install(phy, mac, staNodes);

    mac.SetType("ns3::ApWifiMac", "Ssid", SsidValue(ssid));
    NetDeviceContainer apDevice = wifi.Install(phy, mac, apNode);

    // Mobility Model (Speed: 1, 5, 10 m/s)
    MobilityHelper mobility;
    mobility.SetPositionAllocator("ns3::GridPositionAllocator", "MinX", DoubleValue(0.0), "MinY
    mobility.SetMobilityModel("ns3::ConstantVelocityMobilityModel");
    mobility.Install(staNodes);
    mobility.Install(apNode);

    // Set mobile speed to 5 m/s along X-axis
```

```
    for (uint32_t i = 0; i < staNodes.GetN(); i++) {
        staNodes.Get(i)->GetObject<ConstantVelocityMobilityModel>()->SetVelocity(Vector(5.0, 0.
    }

    InternetStackHelper stack;
    stack.Install(apNode);
    stack.Install(staNodes);

    Ipv4AddressHelper address;
    address.SetBase("192.168.1.0", "255.255.255.0");
    address.Assign(apDevice);
    auto staInterfaces = address.Assign(staDevices);

    // CBR Traffic from STA 0 to AP
    UdpServerHelper server(9000);
    server.Install(apNode.Get(0)).Start(Seconds(1.0));
    UdpClientHelper client(Ipv4Address("192.168.1.1"), 9000);
    client.SetAttribute("MaxPackets", UintegerValue(1000));
    client.Install(staNodes.Get(0)).Start(Seconds(2.0));

    FlowMonitorHelper flowmon;
    Ptr<FlowMonitor> monitor = flowmon.InstallAll();

    Simulator::Stop(Seconds(10.0));
    Simulator::Run();

    for (auto const& [id, s] : monitor->GetFlowStats()) {
        double pdr = (double)s.rxPackets / s.txPackets * 100.0;
        std::cout << "Mobility PDR: " << pdr << "%\n";
    }
    Simulator::Destroy();
    return 0;
}
```

5-Minute Exam Checklist

If Exam Question Asks	What to Write / Remember
TCP Client-Server	`socket -> bind -> listen -> accept` (Server) / `socket -> connect` (Client).
UDP Sockets	`socket(AF_INET, SOCK_DGRAM)` + `recvfrom` & `sendto`.
Timer / Retransmission	`setsockopt(sfd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv))` in a while-loop.
HTTP Webpage Download	`GET /index.html HTTP/1.1\r\nHost: localhost\r\nConnection: close\r\n\r\n` to port 8000.
DNS Resolution	Array of structs `domain -> ip`. Client checks local cache first; if miss, queries UDP server.
Checksum / CRC	1's complement addition with end-around carry / Shift-and-XOR modulo-2 division.
ARP Simulation	Array of structs `ip -> mac`. Client requests MAC for given IP over UDP.
NS-3 Throughput & PDR	`FlowMonitorHelper flowmon; Ptr<FlowMonitor> m = flowmon.InstallAll();`
NS-3 Wireless & Mobility	`WifiHelper` + `MobilityHelper` with `ConstantVelocityMobilityModel`.